KIROWEEN 2026 · ENGLISH

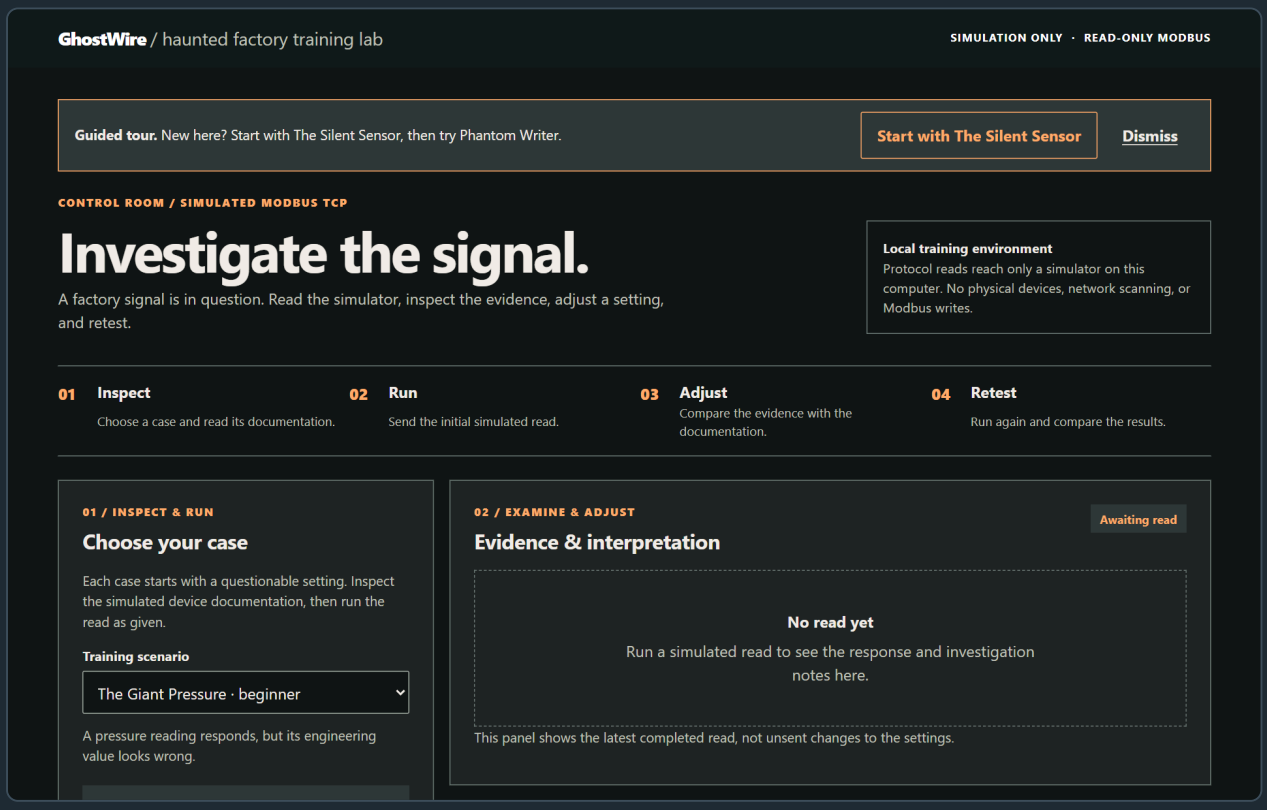
GhostWire

User and Technical Manual

Version 1.1.1 · 13 November 2026

https://github.com/Wayan123/ghostwire-modbus-lab

Apache-2.0



Contents

About this manual

- Who it is for
- The five cases at a glance

Part A · User guide

1. Install and start
2. The screen at a glance
3. How every case works
4. Case 1: The Giant Pressure
5. Wire evidence: reading the bytes
6. Case 2: The Silent Sensor
7. Case 3: The Wrong Register
8. Case 4: Phantom Writer (security incident)
9. Case 5: The Night Scanner (security incident)
10. Case history and report
11. Phone and narrow screens
12. Optional AI explanation
13. Troubleshooting

Part B · Technical reference

14. Architecture
15. Modbus TCP frame format
16. How the incident logs are produced
17. Safety boundaries
18. HTTP API (local)
19. Verification
20. License and credits

About this manual

GhostWire is an OT (operational technology) security and troubleshooting lab. It teaches technicians and students to investigate Modbus TCP faults and a security incident **from evidence**: real TCP exchanges with simulated devices on their own computer.

Everything is simulated. GhostWire never connects to physical equipment and never scans networks, and learners cannot send Modbus writes.

This manual has two parts:

- **Part A, user guide:** installing, the screen layout, and a step-by-step walkthrough of all five cases.
- **Part B, technical reference:** architecture, the Modbus TCP frame format, how the incident logs are produced, safety boundaries, the API, and verification.

Every screen in this manual was captured from GhostWire v1.1.0 running on Windows.

Who it is for

Person	What they learn
Technicians and apprentices	Read a Modbus register, interpret raw values, scale and units, and fix common configuration errors
Students (automation, ICS security)	How a Modbus TCP frame is built, byte by byte
OT security trainees	Investigate an unauthorised write and a unit-ID sweep from a traffic log
Trainers	Run repeatable, safe exercises without any lab hardware

The five cases at a glance

Case	Type	Symptom	Lesson
The Giant Pressure	Troubleshooting, beginner	The raw value 39 is shown as 39 bar	The scale must come from the documentation (0.01)
The Silent Sensor	Troubleshooting, beginner	The read times out	The unit ID is wrong (17); a timeout alone proves nothing
The Wrong Register	Troubleshooting, intermediate	Exception 02, Illegal Data Address	The manual uses zero-based addresses (20, not 21)
Phantom Writer	Security incident, intermediate	The setpoint reads 95 °C instead of 65 °C	Find the one FC06 write in the traffic log
The Night Scanner	Security incident, intermediate	The process looks normal	Find the client that sweeps unit IDs

Part A · User guide

1. Install and start

1.1 Windows, no install (easiest)

1. Download `GhostWire-Windows-v1.1.1-standalone.zip` from https://github.com/Wayan123/ghostwire-modbus-lab/releases/latest. It is about 40 MB and includes the .NET runtime.
2. Unzip it and double-click `GhostWire.exe`. A console window opens. **Keep it open**, because it is the server.
3. Open http://127.0.0.1:8000 in a browser.
4. Windows SmartScreen may warn because the file is unsigned. Choose **More info → Run anyway**.

A smaller zip (about 130 KB) on the same page needs the .NET 8 ASP.NET Core Runtime (x64) to be installed.

1.2 From source (any OS)

Install the .NET 8 SDK, then run this from the repository root:

```
dotnet run --project csharp/GhostWire/GhostWire.csproj
```

Open http://127.0.0.1:8000 and stop the server with Ctrl+C. To use another port, set `GHOSTWIRE_PORT` (1–65535) before starting.

No hardware, account, AI key or NuGet package is needed.

2. The screen at a glance

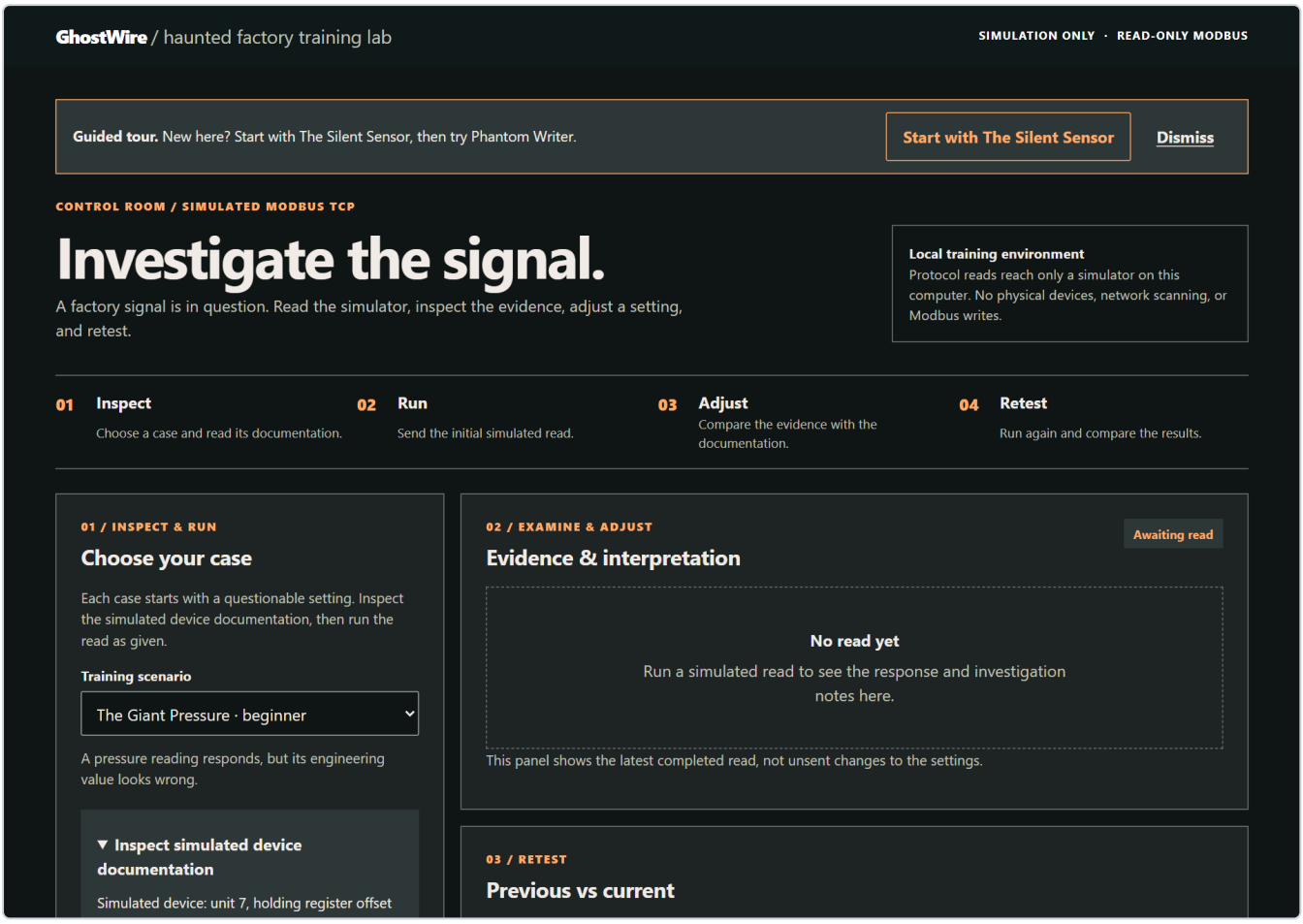


Figure 1. GhostWire on first visit: the guided-tour banner and the case picker.

The page is divided into numbered sections:

Section	Content
01 / Inspect & Run	**Choose your case**: the **Training scenario** picker (grouped into **Troubleshooting** and **Security incidents**, with the difficulty), the scenario summary, **Inspect simulated device documentation**, and the **Read request settings**: **Unit ID**, **Function code** (03 or 04), **Address**, **Quantity**, **Scale multiplier** and **Run simulated read**
02 / Examine & Adjust	**Evidence & interpretation**: the status, engineering value, raw registers, **Observations**, **Possible causes**, **Training hints**, **Next steps**, the **Verification** line, **Wire evidence (simulated frames)** and **Explanation**
02B / Investigate the capture	Security cases only: the **Simulated traffic log** and the answer form
03 / Retest	**Previous vs current**: compares the last two reads after you change a setting
04 / Review	**Case history & report**: every read and answer attempt in this session, and **Print case report**

On a first visit, a **Guided tour** banner suggests starting with The Silent Sensor and then Phantom Writer. **Dismiss** hides it for this browser.

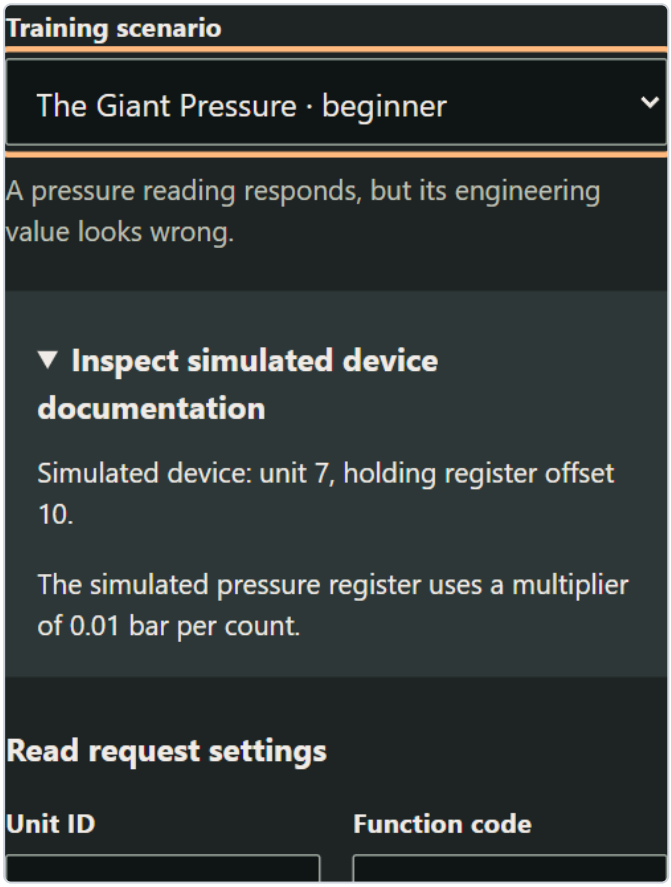
> If a setting is out of range (for example Unit ID 999), nothing is sent, and a message names the allowed range.

3. How every case works

1. **Choose the case** in **Training scenario**.
2. Open **Inspect simulated device documentation**. It is the "manual" of the simulated device, and it is the ground truth.
3. Press **Run simulated read** with the initial settings, and look at the evidence.
4. Compare the evidence with the documentation and change **one** setting.
5. Run the read again. The case is solved only when the **Verification** line says so.

`Verified` becomes true only when deterministic rules confirm that the settings match the documentation **and** the simulator gave a successful response. A plausible-looking number is not enough.

4. Case 1: The Giant Pressure



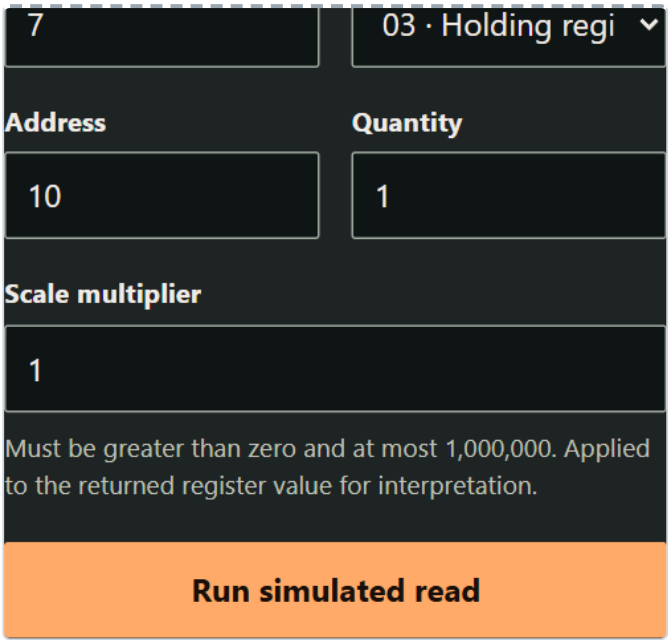


Figure 2. The Giant Pressure: scenario and simulated documentation.

Initial settings: unit 7, FC03, address 10, quantity 1, scale 1.

1. Run the read. The simulator returns raw `39`, and with scale 1 the display shows 39 bar. That is far too high for this sensor.

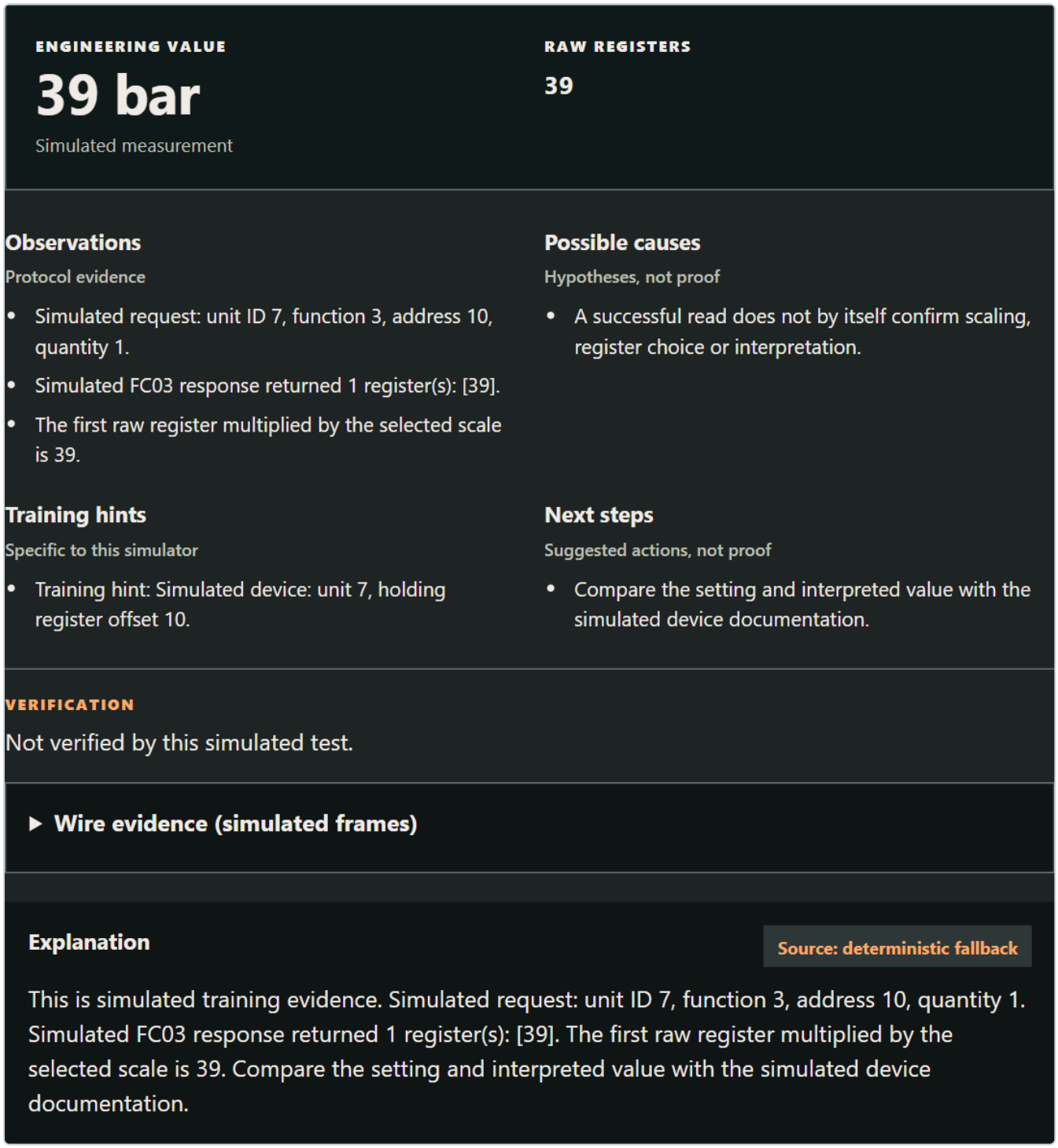


Figure 3. First read: raw 39 shown as 39 bar, not verified.

2. The documentation says the register holds pressure in **0.01 bar** units.
3. Set **Scale multiplier** to `0.01` and run again. The result is `0.39 bar`, verified.

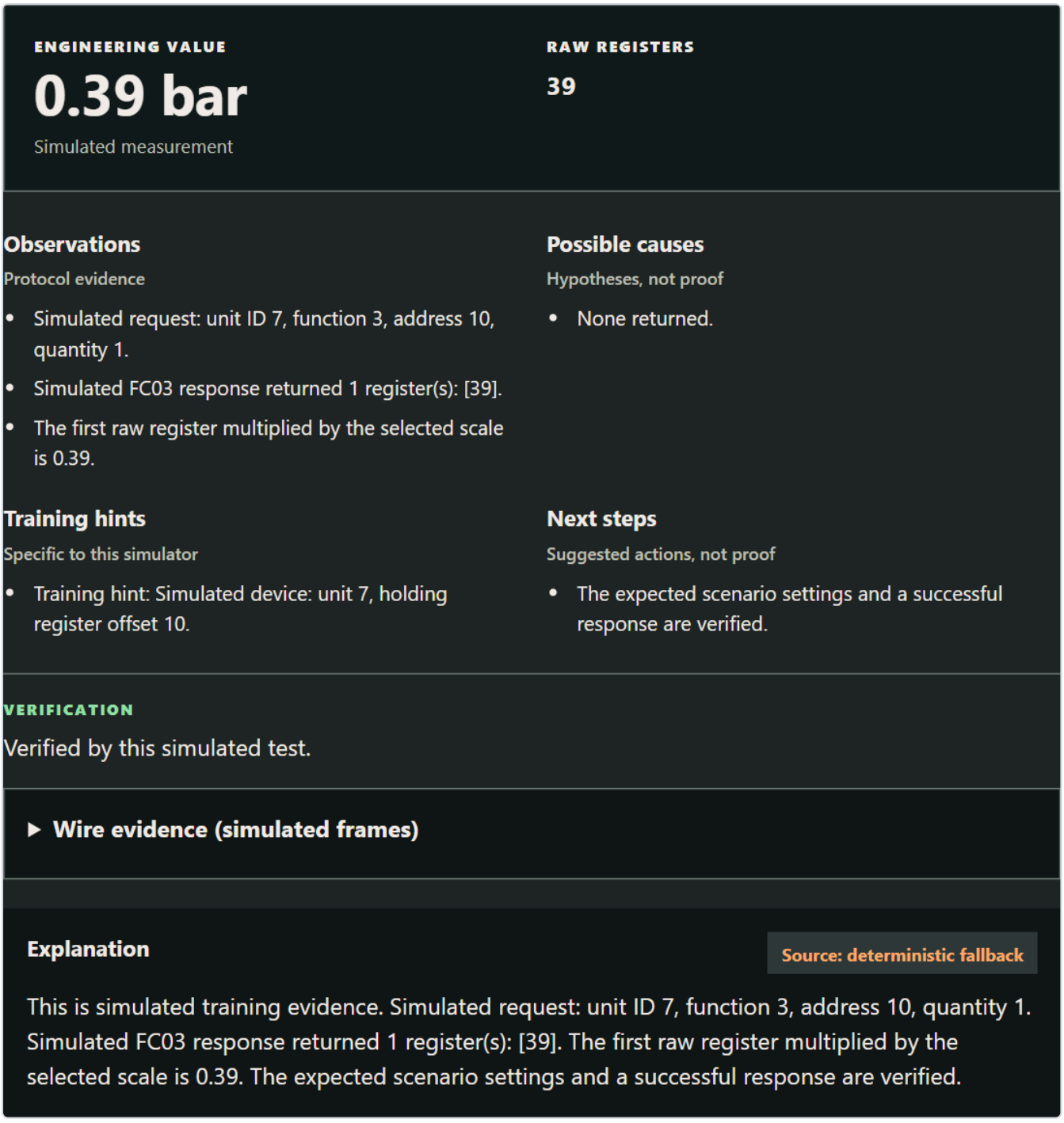
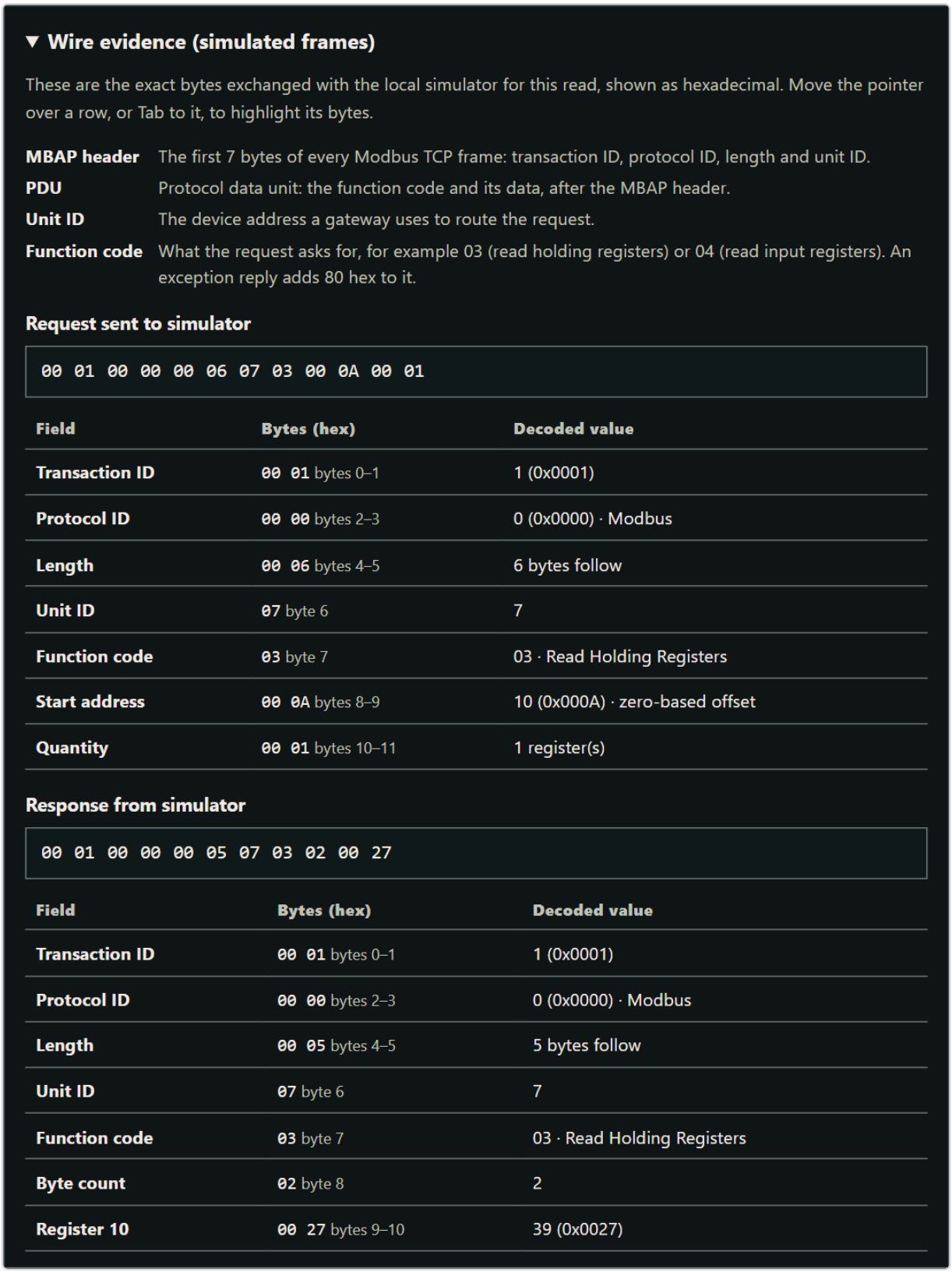


Figure 4. After the fix: 0.39 bar, verified.

Lesson: a Modbus register holds an integer. The engineering value is raw × scale, and the scale always comes from the device manual.

5. Wire evidence: reading the bytes

Open **Wire evidence (simulated frames)** under the Verification line. It is available after every read in every case.



▼ Wire evidence (simulated frames)

These are the exact bytes exchanged with the local simulator for this read, shown as hexadecimal. Move the pointer over a row, or Tab to it, to highlight its bytes.

MBAP header	The first 7 bytes of every Modbus TCP frame: transaction ID, protocol ID, length and unit ID.
PDU	Protocol data unit: the function code and its data, after the MBAP header.
Unit ID	The device address a gateway uses to route the request.
Function code	What the request asks for, for example 03 (read holding registers) or 04 (read input registers). An exception reply adds 80 hex to it.

Request sent to simulator

```
00 01 00 00 00 06 07 03 00 0A 00 01
```

Field	Bytes (hex)	Decoded value
Transaction ID	**00 01** bytes 0–1	1 (0x0001)
Protocol ID	**00 00** bytes 2–3	0 (0x0000) · Modbus
Length	**00 06** bytes 4–5	6 bytes follow
Unit ID	**07** byte 6	7
Function code	**03** byte 7	03 · Read Holding Registers
Start address	**00 0A** bytes 8–9	10 (0x000A) · zero-based offset
Quantity	**00 01** bytes 10–11	1 register(s)

Response from simulator

```
00 01 00 00 00 05 07 03 02 00 27
```

Field	Bytes (hex)	Decoded value
Transaction ID	**00 01** bytes 0–1	1 (0x0001)
Protocol ID	**00 00** bytes 2–3	0 (0x0000) · Modbus
Length	**00 05** bytes 4–5	5 bytes follow
Unit ID	**07** byte 6	7
Function code	**03** byte 7	03 · Read Holding Registers
Byte count	**02** byte 8	2
Register 10	**00 27** bytes 9–10	39 (0x0027)

Figure 5. Wire evidence for the corrected Giant Pressure read: each MBAP and PDU field decoded, with its byte range.

The corrected read sends:

```
00 01  00 00  00 06  07  03  00 0A  00 01
TxID   Proto  Len    Unit FC  Addr   Qty
```

and receives:

```
00 01  00 00  00 05  07  03  02     00 27
TxID   Proto  Len    Unit FC  Bytes  Value (0x0027 = 39)
```

Tab to a field row, or hover over it, to highlight its bytes. Each row also gives the byte range in text, such as "bytes 0–1" for the transaction ID. The panel defines the MBAP header, the PDU, the unit ID and the function code. After a timeout, only the request frame is shown, because no response arrived.

6. Case 2: The Silent Sensor

Initial settings: unit 1, FC04, address 0.

1. Run the read. It **times out**. Wire evidence shows only the request `00 01 00 00 00 06 01 04 00 00 00 01`.

ENGINEERING VALUE

Not returned

Simulated measurement

RAW REGISTERS

Not returned

Observations

Protocol evidence

- Simulated request: unit ID 1, function 4, address 0, quantity 1.
- The simulated Modbus request timed out without a response.

Possible causes

Hypotheses, not proof

- A unit ID mismatch is one possible cause; timeouts alone do not prove it.
- A connection issue or unavailable device could also cause a timeout.

Training hints

Specific to this simulator

- Training hint: Simulated gateway routes this device at unit 17.

Next steps

Suggested actions, not proof

- Compare the unit ID with the simulated gateway documentation and retest.

VERIFICATION

Not verified by this simulated test.

▼ Wire evidence (simulated frames)

These are the exact bytes exchanged with the local simulator for this read, shown as hexadecimal. Move the pointer over a row, or Tab to it, to highlight its bytes.

MBAP header	The first 7 bytes of every Modbus TCP frame: transaction ID, protocol ID, length and unit ID.
PDU	Protocol data unit: the function code and its data, after the MBAP header.
Unit ID	The device address a gateway uses to route the request.
Function code	What the request asks for, for example 03 (read holding registers) or 04 (read input registers). An exception reply adds 80 hex to it.

Request sent to simulator

```
00 01 00 00 00 06 01 04 00 00 00 01
```

Field	Bytes (hex)	Decoded value
Transaction ID	**00 01** bytes 0–1	1 (0x0001)
Protocol ID	**00 00** bytes 2–3	0 (0x0000) · Modbus
Length	**00 06** bytes 4–5	6 bytes follow

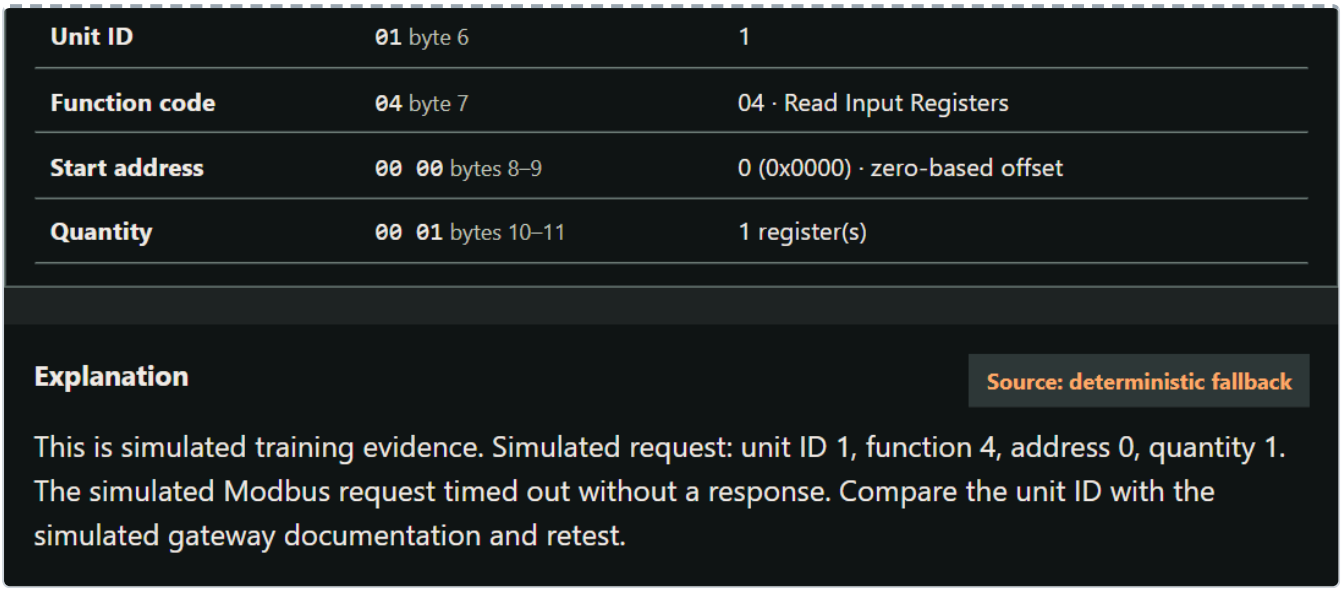


Figure 6. The Silent Sensor: timeout, with possible causes listed. The timeout does not prove a single cause.

2. **Possible causes** lists a wrong unit ID, connection, routing and an unavailable device. All of them remain possible.
3. The documentation gives the device's unit ID as **17**. Set **Unit ID** to `17` and run again. The simulator returns raw `73`, verified.

ENGINEERING VALUE

73

Simulated measurement

RAW REGISTERS

73

Observations

Protocol evidence

- Simulated request: unit ID 17, function 4, address 0, quantity 1.
- Simulated FC04 response returned 1 register(s): [73].
- The first raw register multiplied by the selected scale is 73.

Possible causes

Hypotheses, not proof

- None returned.

Training hints

Specific to this simulator

- Training hint: Simulated gateway routes this device at unit 17.

Next steps

Suggested actions, not proof

- The expected scenario settings and a successful response are verified.

VERIFICATION

Verified by this simulated test.

▼ Wire evidence (simulated frames)

These are the exact bytes exchanged with the local simulator for this read, shown as hexadecimal. Move the pointer over a row, or Tab to it, to highlight its bytes.

MBAP header	The first 7 bytes of every Modbus TCP frame: transaction ID, protocol ID, length and unit ID.
PDU	Protocol data unit: the function code and its data, after the MBAP header.
Unit ID	The device address a gateway uses to route the request.
Function code	What the request asks for, for example 03 (read holding registers) or 04 (read input registers). An exception reply adds 80 hex to it.

Request sent to simulator

```
00 01 00 00 00 06 11 04 00 00 00 01
```

Field	Bytes (hex)	Decoded value
Transaction ID	`00 01` bytes 0–1	1 (0x0001)
Protocol ID	`00 00` bytes 2–3	0 (0x0000) · Modbus

Length	**00 06** bytes 4–5	6 bytes follow
Unit ID	**11** byte 6	17
Function code	**04** byte 7	04 · Read Input Registers
Start address	**00 00** bytes 8–9	0 (0x0000) · zero-based offset
Quantity	**00 01** bytes 10–11	1 register(s)

Response from simulator

```
00 01 00 00 00 05 11 04 02 00 49
```

Field	Bytes (hex)	Decoded value
Transaction ID	**00 01** bytes 0–1	1 (0x0001)
Protocol ID	**00 00** bytes 2–3	0 (0x0000) · Modbus
Length	**00 05** bytes 4–5	5 bytes follow
Unit ID	**11** byte 6	17
Function code	**04** byte 7	04 · Read Input Registers
Byte count	**02** byte 8	2
Register 0	**00 49** bytes 9–10	73 (0x0049)

Explanation Source: deterministic fallback

This is simulated training evidence. Simulated request: unit ID 17, function 4, address 0, quantity 1. Simulated FC04 response returned 1 register(s): [73]. The first raw register multiplied by the selected scale is 73. The expected scenario settings and a successful response are verified.

Figure 7. After setting unit 17: raw 73, verified.

Lesson: the timeout did not prove that the unit ID was wrong. The successful retest with 17 is what verifies it.

7. Case 3: The Wrong Register

Initial settings: unit 3, FC04, address 21, quantity 1.

1. Run the read. The device answers with an **exception**. The response `00 01 00 00 00 03 03 84 02` decodes as function `84` (exception for 04) and exception code `02`, Illegal Data Address.

ENGINEERING VALUE

Not returned

Simulated measurement

RAW REGISTERS

Not returned

Observations

Protocol evidence

- Simulated request: unit ID 3, function 4, address 21, quantity 1.
- The simulated device returned Modbus exception code 02.

Possible causes

Hypotheses, not proof

- The requested register range may not exist for this function.

Training hints

Specific to this simulator

- Training hint: Simulated device: unit 3, input register offset 20.

Next steps

Suggested actions, not proof

- Compare function code and zero-based address with the simulated device documentation.

VERIFICATION

Not verified by this simulated test.

▼ Wire evidence (simulated frames)

These are the exact bytes exchanged with the local simulator for this read, shown as hexadecimal. Move the pointer over a row, or Tab to it, to highlight its bytes.

MBAP header	The first 7 bytes of every Modbus TCP frame: transaction ID, protocol ID, length and unit ID.
PDU	Protocol data unit: the function code and its data, after the MBAP header.
Unit ID	The device address a gateway uses to route the request.
Function code	What the request asks for, for example 03 (read holding registers) or 04 (read input registers). An exception reply adds 80 hex to it.

Request sent to simulator

```
00 01 00 00 00 06 03 04 00 15 00 01
```

Field	Bytes (hex)	Decoded value
Transaction ID	**00 01** bytes 0–1	1 (0x0001)
Protocol ID	**00 00** bytes 2–3	0 (0x0000) · Modbus
Length	**00 06** bytes 4–5	6 bytes follow

Unit ID	`03` byte 6	3
Function code	`04` byte 7	04 · Read Input Registers
Start address	`00 15` bytes 8–9	21 (0x0015) · zero-based offset
Quantity	`00 01` bytes 10–11	1 register(s)

Response from simulator

```
00 01 00 00 00 03 03 84 02
```

Field	Bytes (hex)	Decoded value
Transaction ID	`00 01` bytes 0–1	1 (0x0001)
Protocol ID	`00 00` bytes 2–3	0 (0x0000) · Modbus
Length	`00 03` bytes 4–5	3 bytes follow
Unit ID	`03` byte 6	3
Function code	`84` byte 7	84 · Exception for 04
Exception code	`02` byte 8	02 · Illegal Data Address

Explanation Source: deterministic fallback

This is simulated training evidence. Simulated request: unit ID 3, function 4, address 21, quantity 1. The simulated device returned Modbus exception code 02. Compare function code and zero-based address with the simulated device documentation.

Figure 8. The Wrong Register: exception 02, Illegal Data Address, decoded in the wire evidence.

2. The training hint and the documentation give the register as unit 3, input register at **zero-based** offset 20.
3. Set **Address** to `20` and run again. The simulator returns raw `125`, verified.

ENGINEERING VALUE

125

Simulated measurement

RAW REGISTERS

125

Observations

Protocol evidence

- Simulated request: unit ID 3, function 4, address 20, quantity 1.
- Simulated FC04 response returned 1 register(s): [125].
- The first raw register multiplied by the selected scale is 125.

Possible causes

Hypotheses, not proof

- None returned.

Training hints

Specific to this simulator

- Training hint: Simulated device: unit 3, input register offset 20.

Next steps

Suggested actions, not proof

- The expected scenario settings and a successful response are verified.

VERIFICATION

Verified by this simulated test.

▼ Wire evidence (simulated frames)

These are the exact bytes exchanged with the local simulator for this read, shown as hexadecimal. Move the pointer over a row, or Tab to it, to highlight its bytes.

MBAP header	The first 7 bytes of every Modbus TCP frame: transaction ID, protocol ID, length and unit ID.
PDU	Protocol data unit: the function code and its data, after the MBAP header.
Unit ID	The device address a gateway uses to route the request.
Function code	What the request asks for, for example 03 (read holding registers) or 04 (read input registers). An exception reply adds 80 hex to it.

Request sent to simulator

```
00 01 00 00 00 06 03 04 00 14 00 01
```

Field	Bytes (hex)	Decoded value
Transaction ID	**00 01** bytes 0–1	1 (0x0001)
Protocol ID	**00 00** bytes 2–3	0 (0x0000) · Modbus

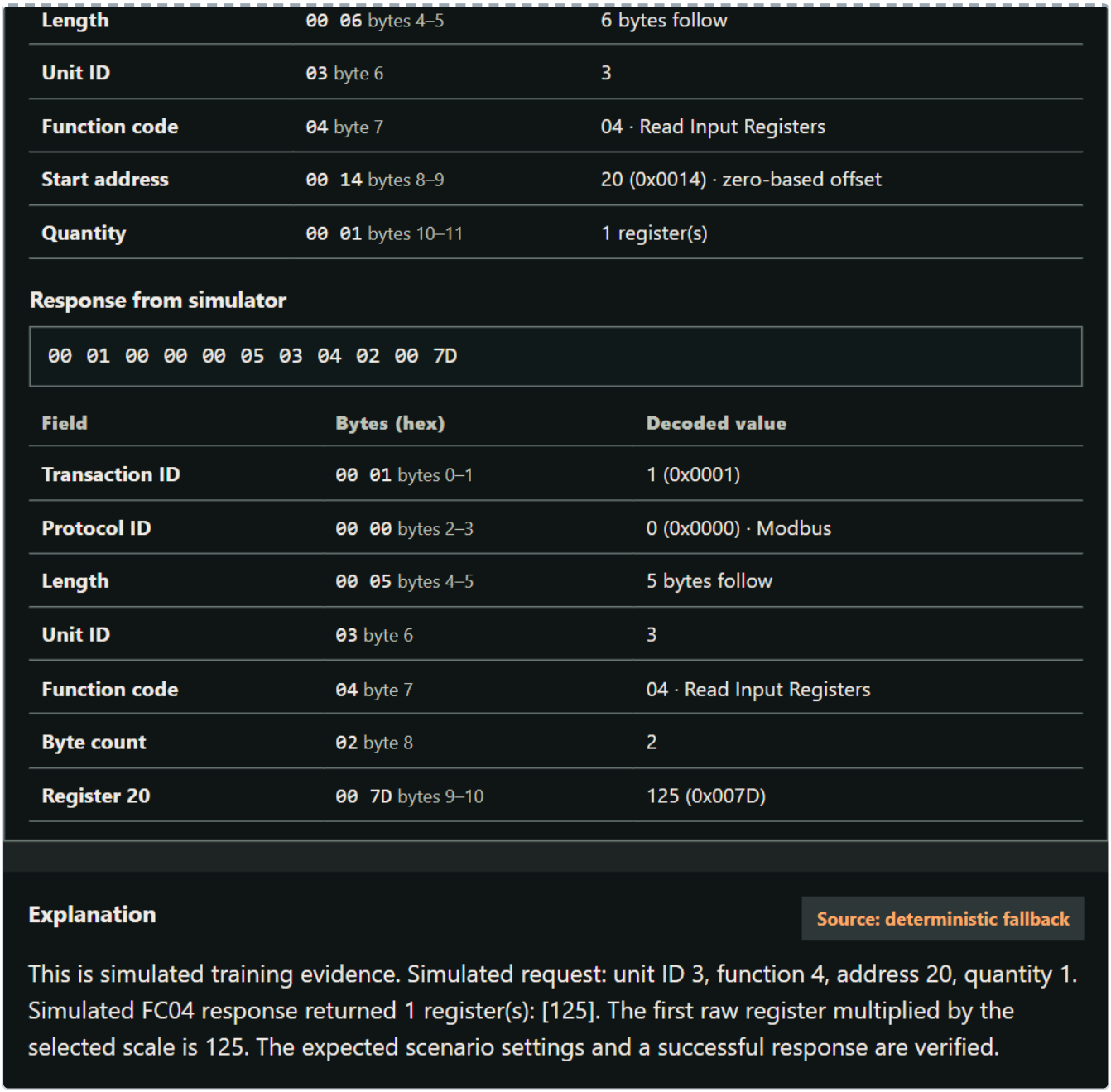


Figure 9. After using address 20: raw 125, verified.

Lesson: manuals use different address conventions. "Register 21" in a one-based list is offset 20 on the wire.

8. Case 4: Phantom Writer (security incident)

Initial settings: unit 5, FC03, address 100, quantity 1, scale 0.1.

1. Run the read. The simulator returns raw `950`, shown as 95 °C. The documented normal setpoint is 650 (65.0 °C). The read confirms the current value, but it does not show who changed it.
2. Scroll to **02B / Investigate the capture**, the **Simulated traffic log**. It lists 12 captured transactions. Times show as HH:MM:SS (UTC), and hovering a time shows the full timestamp. **Show frames** on a row displays its bytes.

02B / INVESTIGATE THE CAPTURE

Simulated traffic log

Simulated capture. Simulated capture generated at server start from loopback exchanges with a private simulator. Modbus TCP frames carry no client identity; client labels are capture metadata.

FC03 (Read Holding Registers) reads holding registers. FC06 (Write Single Register) changes one holding register. The unit ID is the device address a gateway routes the request to. Status shows what came back: a response, an exception code, or a timeout (no reply).

Find the entry that changed the setpoint, then type its transaction ID and client label into the answer form below.

Scroll the table sideways to see every column.

Simulated capture, one row per Modbus transaction (times in UTC)

Seq	Simulated time	Client label	Transaction ID	Unit ID	Function	Status
1	02:13:00	HMI-01	1	5	03 · Read Holding Registers	Response
2	02:13:05	HMI-01	2	5	03 · Read Holding Registers	Response
3	02:13:10	HMI-01	3	5	03 · Read Holding Registers	Response
4	02:13:15	HMI-01	4	5	03 · Read Holding Registers	Response
5	02:13:20	HMI-01	5	5	03 · Read Holding Registers	Response
6	02:13:25	HMI-01	6	5	03 · Read Holding Registers	Response
7	02:13:30	HMI-01	7	5	03 · Read Holding Registers	Response
8	02:13:33	MAINT-LAPTOP-07	1111	5	06 · Write Single Register	Response
9	02:13:35	HMI-01	8	5	03 · Read Holding Registers	Response
10	02:13:40	HMI-01	9	5	03 · Read Holding Registers	Response
11	02:13:45	HMI-01	10	5	03 · Read Holding Registers	Response
12	02:13:50	HMI-01	11	5	03 · Read Holding Registers	Response

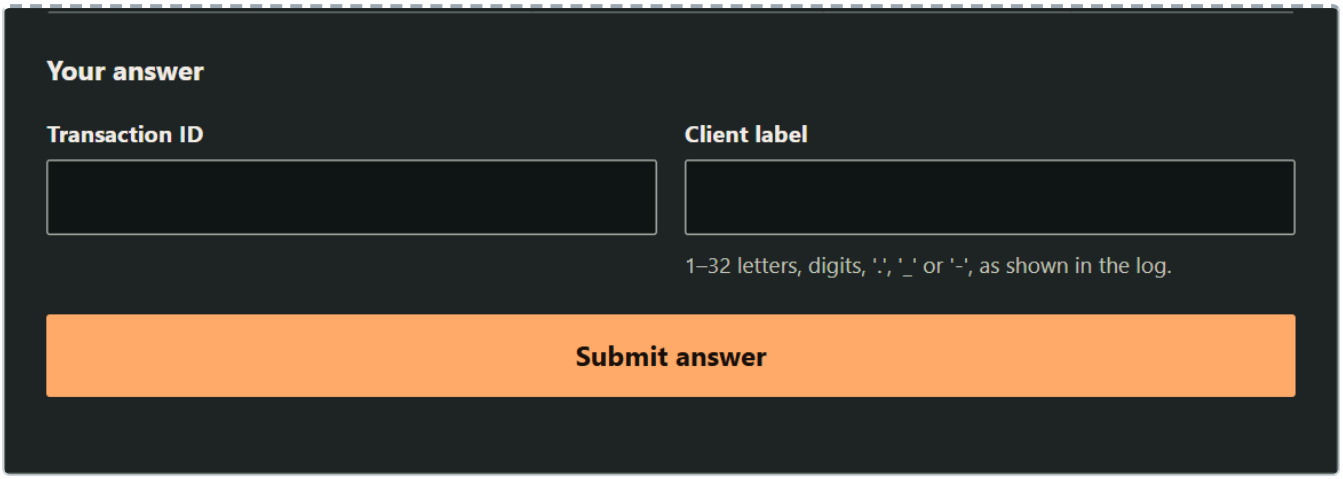


Figure 10. Phantom Writer: the incident task, the 12-row simulated traffic log and the answer form.

3. Find the entry that changed the setpoint: the one **FC06 (Write Single Register)** row. Note its transaction ID and client label.
4. Type them into **Transaction ID** and **Client label**, then press **Submit answer**.

A wrong pair returns "not verified" **without revealing the correct answer**. After 3 wrong answers a training hint appears, and a second one after the 4th. The hints never name the entry.

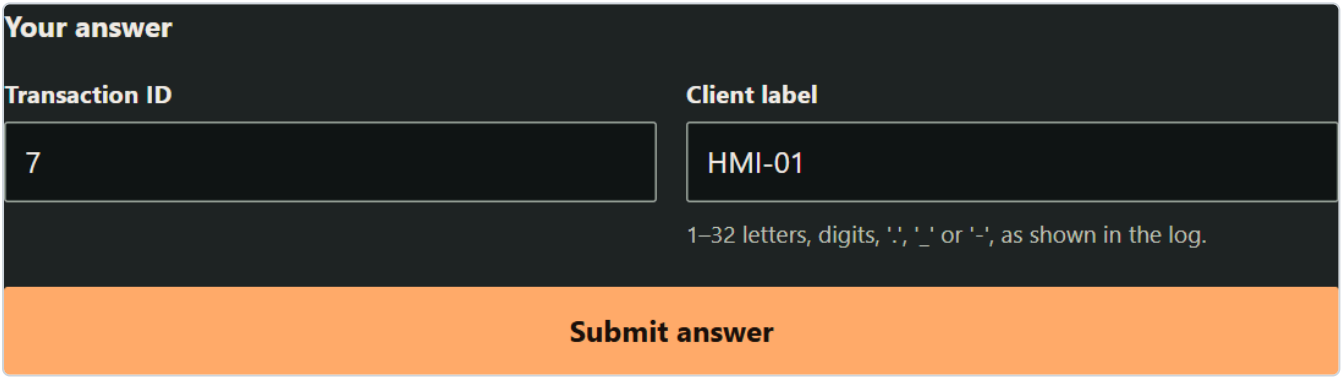


Figure 11. A wrong answer: not verified, with no hint about the right row.

Verified answer: transaction `1111`, client `MAINT-LAPTOP-07`, at `2026-10-31T02:13:33Z`. This FC06 write changed register 100 from 650 to 950. The read before it (transaction 7) returned 650, and the read after it (transaction 8) returned 950.

02B / INVESTIGATE THE CAPTURE

Simulated traffic log

Simulated capture. Simulated capture generated at server start from loopback exchanges with a private simulator. Modbus TCP frames carry no client identity; client labels are capture metadata.

FC03 (Read Holding Registers) reads holding registers. FC06 (Write Single Register) changes one holding register. The unit ID is the device address a gateway routes the request to. Status shows what came back: a response, an exception code, or a timeout (no reply).

Find the entry that changed the setpoint, then type its transaction ID and client label into the answer form below.

Scroll the table sideways to see every column.

Simulated capture, one row per Modbus transaction (times in UTC)

Seq	Simulated time	Client label	Transaction ID	Unit ID	Function
1	02:13:00	HMI-01	1	5	03 · Read Holding Registers
2	02:13:05	HMI-01	2	5	03 · Read Holding Registers
3	02:13:10	HMI-01	3	5	03 · Read Holding Registers
4	02:13:15	HMI-01	4	5	03 · Read Holding Registers
5	02:13:20	HMI-01	5	5	03 · Read Holding Registers
6	02:13:25	HMI-01	6	5	03 · Read Holding Registers
7	02:13:30	HMI-01	7	5	03 · Read Holding Registers
8 ⚠ Unauthorized write	02:13:33	MAINT-LAPTOP-07	1111	5	06 · Write Single Register
9	02:13:35	HMI-01	8	5	03 · Read Holding Registers
10	02:13:40	HMI-01	9	5	03 · Read Holding Registers
11	02:13:45	HMI-01	10	5	03 · Read Holding Registers
12	02:13:50	HMI-01	11	5	03 · Read Holding Registers

Your answer

Transaction ID

1111

Client label

MAINT-LAPTOP-07

1–32 letters, digits, '.', '_' or '-', as shown in the log.

Submit answer

Status: answer verified

Observations

From the simulated capture

- In this simulated capture, transaction 1111 at 2026-10-31T02:13:33Z from client label MAINT-LAPTOP-07 is function 06 (Write Single Register) to unit 5, register 100, value 950.
- Old value: the last simulated read before the write (sequence 7, transaction 7) returned 650 at register 100.
- New value: the simulated write echo returned 950, and the next read (sequence 9, transaction 8) returned 950.
- The other entries in this simulated capture come from client label(s): HMI-01.

Possible causes

Hypotheses, not proof

- None returned.

Training hints

Specific to this simulator

- Training hint: Modbus TCP has no built-in authentication, so any client that can reach the device can send FC06 in this simulation.

Next steps

Suggested actions, not proof

- Compare the write time with approved change records.

Next steps · real-world guidance, not verified by this simulation

- Real-world guidance, not verified by this simulation: segment the control network into IEC 62443 zones and conduits so only approved systems can reach the controller.
- Real-world guidance, not verified by this simulation: allow-list write function codes (such as FC06 and FC16) per source on a Modbus-aware gateway or industrial firewall.
- Real-world guidance, not verified by this simulation: where devices support it, use Modbus/TCP Security (TLS with X.509 certificates that carry roles) so writes require an authorized identity.
- Real-world guidance, not verified by this simulation: log setpoint changes with source and time, and review them against change records.

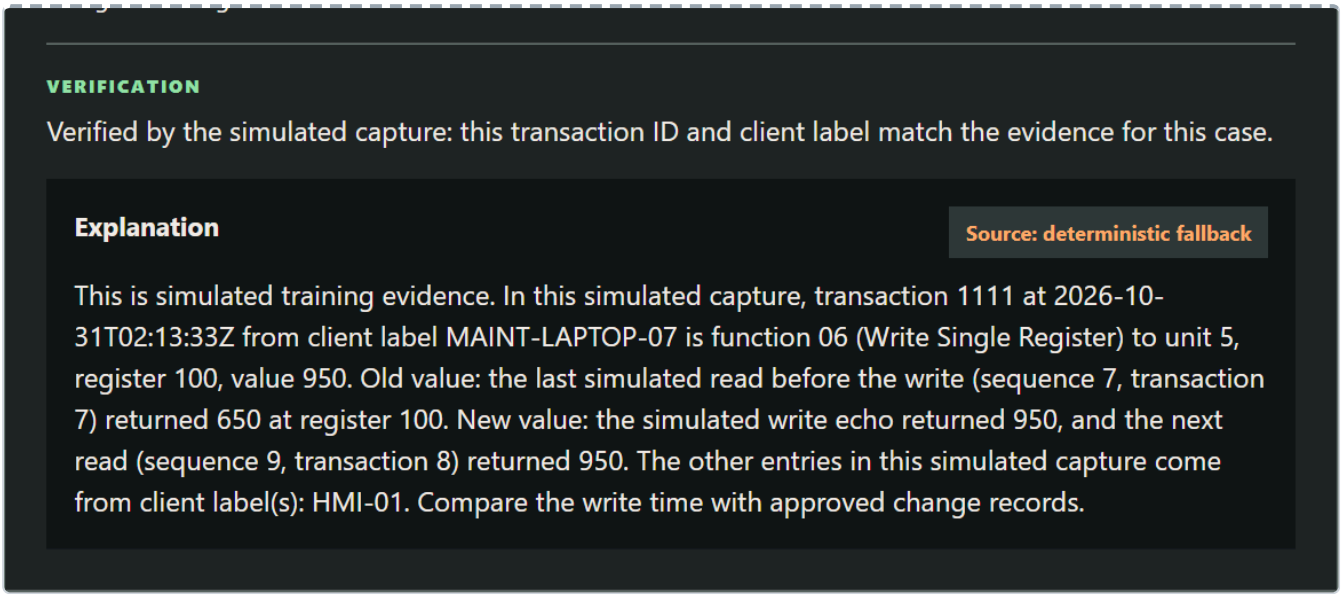


Figure 12. Verified: the write row is marked "⚠ Unauthorized write" and four mitigations appear.

After verification, the row is marked ⚠ **Unauthorized write**, and four mitigations appear under **Next steps · real-world guidance, not verified by this simulation**.

9. Case 5: The Night Scanner (security incident)

Initial settings: unit 5, FC03, address 100, scale 0.1.

1. Run the read. It returns raw `650` (65 °C), the documented normal value, and is verified. The process looks fine. A normal read does not show which other clients queried the gateway.
2. Open the **Simulated traffic log** (15 transactions). The **Status** column shows each result as text: "Response", "Timeout (no reply)" or "Exception 02 · Illegal Data Address". `HMI-01` polls unit 5 every 5 s (transactions 1–5).
3. Find the client that asks **many different unit IDs**, and submit the transaction ID of its **first** request and its client label.

Hint 1 (after 3 misses): "Look at the Unit ID column: does any client ask many different units?" Hint 2 (after 4 misses): "Find the first request in that run."

Verified answer: client `ENG-WS-13`, transaction `301` at `2026-11-01T03:12:07Z`. It sent 10 FC03 reads of address 0 to unit IDs 1–10, one second apart (transactions 301–310):

- 8 requests timed out;
- unit 5 returned exception 02, because it has no register 0;
- unit 9, which the documentation does not list, answered with raw 1.

02B / INVESTIGATE THE CAPTURE

Simulated traffic log

Simulated capture. Simulated capture generated at server start from loopback exchanges with a private gateway simulator. Modbus TCP frames carry no client identity; client labels are capture metadata. A request with no reply within 100 ms is logged as a timeout.

FC03 (Read Holding Registers) reads holding registers. FC06 (Write Single Register) changes one holding register. The unit ID is the device address a gateway routes the request to. Status shows what came back: a response, an exception code, or a timeout (no reply).

Find the client that queried many different unit IDs, then type the transaction ID of its first request and its client label into the answer form below.

Scroll the table sideways to see every column.

Simulated capture, one row per Modbus transaction (times in UTC)

Seq	Simulated time	Client label	Transaction ID	Unit ID	Function	Status
1	03:12:00	HMI-01	1	5	03 · Read Holding Registers	Respons
2	03:12:05	HMI-01	2	5	03 · Read Holding Registers	Respons
3 ⚠ First scan request	03:12:07	ENG-WS-13	301	1	03 · Read Holding Registers	Timeout
4	03:12:08	ENG-WS-13	302	2	03 · Read Holding Registers	Timeout
5	03:12:09	ENG-WS-13	303	3	03 · Read Holding Registers	Timeout
6	03:12:10	HMI-01	3	5	03 · Read Holding Registers	Respons
7	03:12:10	ENG-WS-13	304	4	03 · Read Holding Registers	Timeout
8	03:12:11	ENG-WS-13	305	5	03 · Read Holding Registers	Exceptio Data Ad
9	03:12:12	ENG-WS-13	306	6	03 · Read Holding Registers	Timeout
10	03:12:13	ENG-WS-13	307	7	03 · Read Holding Registers	Timeout
11	03:12:14	ENG-WS-13	308	8	03 · Read Holding Registers	Timeout

12	03:12:15	HMI-01	4	5	03 · Read Holding Registers	Respons
13	03:12:15	ENG-WS-13	309	9	03 · Read Holding Registers	Respons
14	03:12:16	ENG-WS-13	310	10	03 · Read Holding Registers	Timeout
15	03:12:20	HMI-01	5	5	03 · Read Holding Registers	Respons

Your answer

Transaction ID

301

Client label

ENG-WS-13

1–32 letters, digits, '.', '_' or '-', as shown in the log.

Submit answer

Status: answer verified

Observations

From the simulated capture

- In this simulated capture, client label ENG-WS-13 sent 10 requests to unit IDs 1 to 10 in increasing order, using function(s) FC03 at address(es) 0.
- The run starts at transaction 301 (2026-11-01T03:12:07Z) and ends at transaction 310 (2026-11-01T03:12:16Z).
- Intervals between consecutive requests from ENG-WS-13: 1 s shortest, 1 s longest.
- Results of these requests: 8 timeout(s), 1 exception(s) (unit 5: exception 02 (Illegal Data Address)), 1 response(s) (from unit 9).
- Client label HMI-01 sent 5 request(s) in this simulated capture, to unit ID(s) 5 only.

Possible causes

Hypotheses, not proof

- None returned.

Training hints

Specific to this simulator

- Training hint: in this simulation, a run of reads to increasing unit IDs from one client, mostly unanswered, is a reconnaissance pattern, not normal

Next steps

Suggested actions, not proof

- Compare unit 9, which answered, with the simulated device documentation, which lists only unit 5.
- Check whether ENG-WS-13 had an approved reason

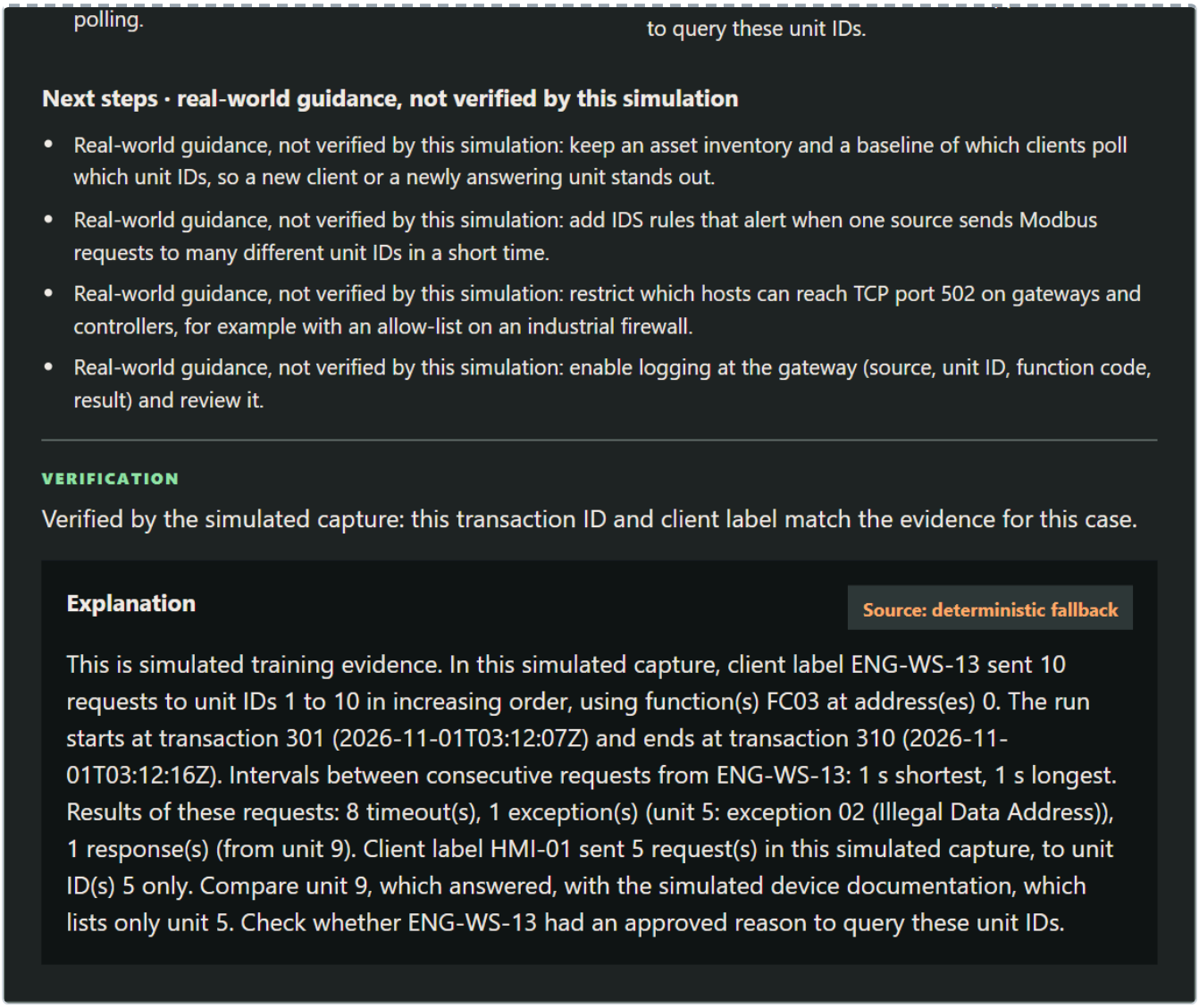


Figure 13. Verified: the first scan request is marked and four sweep-specific mitigations appear.

The mitigations shown are: an asset inventory and a polling baseline; IDS rules for unit-ID sweeps; restricting which hosts can reach port 502; and logging at the gateway. Reading unit 9 or unit 1 through the read form gives the same response or timeout as the log.

10. Case history and report

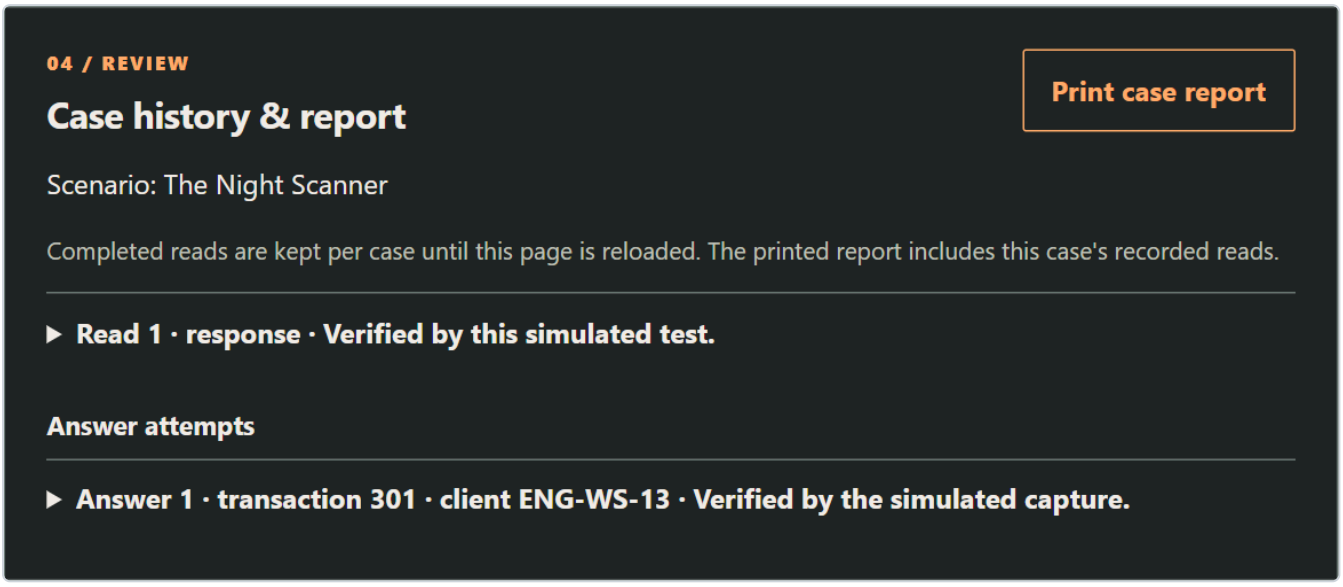


Figure 14. Case history: every read and answer attempt in this session.

04 / Review (Case history & report) keeps each read (settings, status, values) and each answer attempt until the page is reloaded. **03 / Retest** (Previous vs current) compares the last two reads. **Print case report** opens the browser's print dialog, where you can print the report or save it as a PDF.

11. Phone and narrow screens

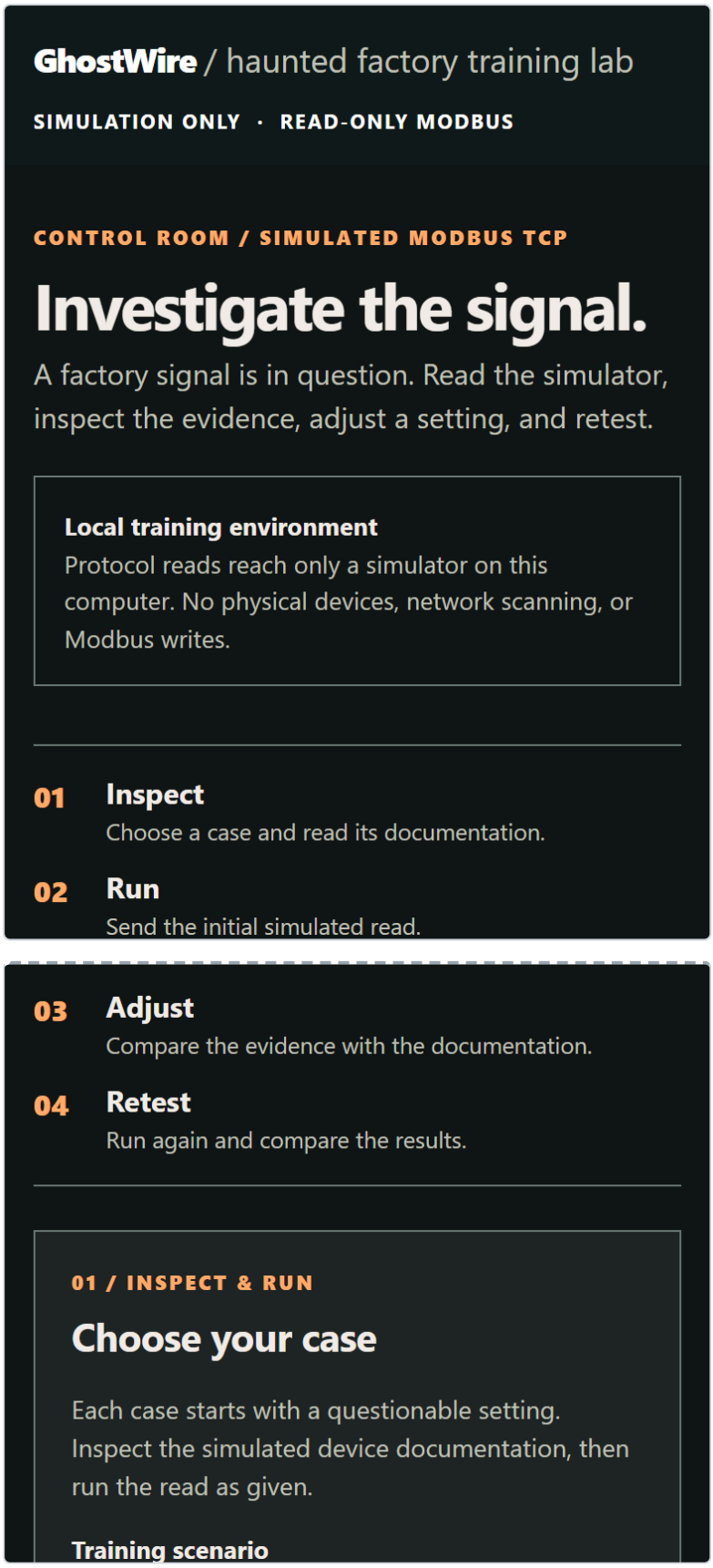


Figure 15. GhostWire at phone width.

The layout adapts to narrow screens. The traffic log table scrolls sideways inside its own box.

12. Optional AI explanation

The **Explanation** panel normally shows a labelled, deterministic explanation. To use an OpenAI-compatible provider instead, set these in the **server environment** before starting:

```
GHOSTWIRE_AI_BASE_URL   (an OpenAI-compatible API base; /chat/completions is appended)
GHOSTWIRE_AI_MODEL
GHOSTWIRE_AI_API_KEY
```

The structured, simulated evidence (including frames) is sent to that provider, so review its terms first. The key stays on the server and is never returned. The server applies a 4 s deadline, ignores redirects, and caps the provider's reply. Provider text is labelled with its source, can be wrong, and never changes the protocol result. If the settings are missing or the call fails, the deterministic text is shown.

13. Troubleshooting

Problem	What to do
The page does not open	Check that the console window is still open and shows "Now listening on: http://127.0.0.1:8000"
The port is already in use	Set `GHOSTWIRE_PORT` to another port and open that port
Error 429	More than 120 requests per minute or 8 tests at once; wait a moment
"Retry loading scenarios" appears	The server stopped or restarted; start it again and press the button
The small zip does not start	Install the .NET 8 ASP.NET Core Runtime x64, or use the standalone zip

Part B · Technical reference

14. Architecture

```
Browser (frontend/: HTML, CSS, JS)
     │  HTTP on 127.0.0.1:8000
     ▼
ASP.NET Core server (C#, .NET 8)
  Program.cs  – HTTP hosting, Host/Origin checks, limits
  Training.cs – cases, validation, diagnosis, optional AI explanation
  Modbus.cs   – Modbus TCP client and simulators (FC03/FC04)
     │  Modbus TCP over loopback, ephemeral ports
     ▼
Read-only simulators, one per case (answer FC06 with exception 01)

At startup only:
  private write-enabled simulator ← scripted FC06 → captured bytes → Phantom Writer log
  private read-only gateway       ← scripted FC03 sweep → captured bytes → Night Scanner log
  (both are disposed before the server accepts requests)
```

There are no third-party NuGet packages. The frontend is plain HTML, CSS and JavaScript, with no build step.

15. Modbus TCP frame format

Bytes	Field	Meaning
0–1	Transaction ID	Matches a response to its request
2–3	Protocol ID	Always `00 00` for Modbus
4–5	Length	The number of bytes that follow (unit + PDU)
6	Unit ID	The device behind a gateway
7	Function code	03 Read Holding Registers, 04 Read Input Registers; +0x80 marks an exception
8…	Data	The request: address and quantity. The response: a byte count and the values, or an exception code

Exception codes used in the cases: `01` Illegal Function and `02` Illegal Data Address.

16. How the incident logs are produced

The traffic logs are **not hand-written**. At startup, under a 5 s overall deadline:

- **Phantom Writer:** a scripted client sends reads and one FC06 to a private, write-enabled loopback simulator. The bytes actually sent and received are decoded into the 12-row log.
- **Night Scanner:** 15 FC03 steps run against a private, read-only gateway simulator that holds unit 5 (register 100 = 650) and unit 9 (register 0 = 1). `HMI-01` polls unit 5 every 5 simulated seconds. `ENG-WS-13` reads address 0 from unit IDs 1–10, one per simulated second. A timeout is logged only when the 100 ms deadline expired and the simulator recorded that it sent nothing.

Both simulators are disposed before the server accepts any HTTP request. Modbus TCP frames carry no client identity, so the client labels are capture metadata.

17. Safety boundaries

Boundary	How it is enforced
No physical equipment	The server listens on loopback only; requests cannot choose a host, port or URL
No learner writes	`/api/test` accepts only FC03 and FC04; the client refuses to build any other function code; every learner-reachable simulator answers FC06 with exception 01
No network scanning	The Night Scanner sweep is 10 reads against a private loopback simulator at startup
Browser hardening	Host and Origin are restricted to the local origin
Resource limits	Requests up to 16 KiB; 120 per minute; 8 concurrent tests; excess gets 429
No answer oracle	A wrong answer returns the same "not verified" response regardless of which part is wrong

Simulated results say nothing about the safety or condition of a real installation. The Modbus code is a deliberately narrow FC03/FC04 implementation for training. It is not a general-purpose or safety-certified library.

18. HTTP API (local)

Method and path	Purpose
`GET /api/scenarios`	The list of cases with their summary, documentation, category and difficulty
`POST /api/test`	Run one read: `unit_id`, `function_code` (3 or 4), `address`, `quantity`, `scale`, and the scenario ID
`GET /api/incident/{scenario_id}/log`	The simulated traffic log of a security case
`POST /api/incident/answer`	Submit a transaction ID and client label

The responses separate **observations** (status, raw registers, frames, verified) from **interpretation** (possible causes, hints, next steps, explanation). Scenario IDs are `pressure-scaling`, `unit-id`, `register-addressing`, `phantom-writer` and `night-scanner`.

19. Verification

```
dotnet build csharp/GhostWire/GhostWire.csproj
dotnet run --project csharp/GhostWire.Tests/GhostWire.Tests.csproj
```

The test harness uses real loopback TCP and HTTP. It covers all five cases, the frame inspector, the incident logs and answers, input validation, request bounds and the Host/Origin checks. It also runs seeded property-based tests: codec round-trip, decoder totality, transport status, timeout diagnosis, "frames equal wire bytes", and Night Scanner plans. The seed is printed, and `GHOSTWIRE_PBT_SEED` reproduces a run. No external AI provider or physical device is tested. Observed checks are recorded in `docs/verification.md`.

20. License and credits

Apache-2.0. Built for Kiroween 2026 with Kiro: specs in `.kiro/specs`, custom agents, hooks (tests after C# changes, a guard against physical targets, docs sync and a stop check), and a session ledger in `docs/kiro-log/LEDGER.md`. Pre-existing code is disclosed in `docs/existing-code-disclosure.md`.